UNIVERSIDADE da MADEIRA

RELATÓRIO SOBRE O PROJETO Nº2 DE POO

Projeto: Jogo de Aventura em Texto (Sherlock Holmes)

Autores: Lucas Silva 2146224, Mário Belím 2173324, Francisco Gouveia 2178724
Instituição: Universidade da Madeira
Curso: Engenharia Informática
Disciplina: Programação Orientada por Objetos
Docentes: Luís Ferreira , Sergi Bermudez e Yuri Almeida

ÍNDICE

1. Introdução .
1.1. Objetivo do Documento
1.2. Visão Geral do Projeto
2. Descrição Geral
2.1. Objetivo do Projeto
2.2. Requisitos Funcionais
2.3. Tecnologias Utilizadas
2.4. Locais do Jogo
3. Arquitetura do Sistema
3.1. Diagrama de Classes UML
3.2. Descrição Detalhada das Classes
3.3. Aplicação dos Princípios de POO
3.4. Aplicação dos Princípios SOLID
4. Decisões de Projeto e justificações .
4.1. Padrão Command
4.2. Hierarquia de Exceções Personalizadas
4.3. Separação em Pacotes
4.4. Gestão de Pistas vs Inventário
4.5. Desafios Enfrentados
5. Guia para Expansão e Modificação
5.1. Pontos de Extensibilidade
5.2. Adicionando Novo Comando
5.3. Adicionando Novos Locais
5.4. Adicionando Novos Itens
5.5. Boas Práticas para Continuação
5.6. Potenciais Melhorias
6. Testes e Validação
6.1. Estratégia de Testes
6.2. Cenários de Teste Implementados
6.3. Correção de Erros
7. Conclusão
7.1. Resultados Alcançados
7.2. Aprendizagens Obtidas

1.INTRODUÇÃO

1.1. Objetivo do Documento

Este relatório técnico tem como objetivo documentar de forma completa e profissional o processo de
desenvolvimento do jogo de investigação textual "Sherlock Holmes", desenvolvido no âmbito da unidade curricular de Programação Orientada por Objetos (POO) do curso de Engenharia Informática da Universidade da Madeira, no ano letivo 2025/2026.
O documento serve como:
- Guia técnico para compreensão da arquitetura e implementação do sistema;
- Referência para manutenção e expansão futura do projeto;
- Demonstração da aplicação prática dos conceitos de POO e princípios SOLID;
- Registo das decisões técnicas e justificações de design;

1.2. Visão Geral do Projeto

O projeto consiste num jogo de aventura interativo baseado em texto (Text-Based Adventure Game), desenvolvido inteiramente em Java, sem interface gráfica. O jogador assume o papel de Sherlock Holmes, o famoso detetive, e deve resolver um mistério criminal através da exploração de diferentes locais na cidade do
Funchal, recolha de pistas físicas e informações, interação com personagens não jogáveis (NPCs) e uso estratégico de itens descobertos ao longo da investigação.

O jogo apresenta as seguintes características principais:
- Sistema de navegação entre 9 locais distintos interligados;
- Mecanismo de inventário para gestão de itens recolhidos;
- Sistema de pistas separado para organizar informações descobertas;
- Interpretador de comandos robusto que aceita instruções em português;
- Sistema de objetivos que conduz o jogador à resolução do mistério;

Todo o jogo funciona através de comandos textuais digitados pelo utilizador no terminal, criando uma experiência de investigação criminal inspirada nas aventuras de Sherlock Holmes.

2. DESCRIÇÃO GERAL

2.1. Objetivo do Projeto

O sistema implementa um motor de jogo completo capaz de gerir todos os aspetos de um jogo de aventura criminal textual, incluindo:

- Componentes principais implementados:

- Motor de jogo com loop principal de execução.
- Sistema de mundo composto por 9 locais interligados formando um mapa navegável;
- Gestão de estado do jogador
- Mecanismo de itens com diferentes tipos e funcionalidades;
- NPCs com falas únicas e informações únicas;
- Sistema de pistas que guia o jogador na resolução do mistério;
- Tratamento robusto de exceções e erros de input.

- **Limitações do projeto:**

- Jogo funciona exclusivamente em modo texto (terminal/consola);
- Não possui sistema de pontuação;
- Limitado a uma única história;

2.2. Requisitos Funcionais

O sistema implementa todos os requisitos obrigatórios especificados no enunciado do projeto**:**

- **Sistema de Navegação:** O jogador deve poder mover-se entre diferentes locais (Zona velha, Mercado dos lavradores, entrada porto, porto industrial, painel, Armazem 4, portões da quinta, corredor, gabinete).
- **Observação do Ambiente:** O jogador deve poder observar detalhadamente o ambiente onde se encontra (Nome do local, descrição do ambiente, NPCs presentes, lista de itens visíveis no local e Saídas disponíveis).
- **Inspeção de Objetos:** O jogador pode inspecionar objetos específicos para descobrir pistas (revelação de pistas escondidas, adição automática de pistas ao caderno de notas, uma mensagem quando o objeto não existe ou não pode ser inspecionado).
- **Interação com NPCs:** O jogador pode conversar com personagens não jogáveis (NPCs fornecem pistas e informações relevantes, diálogos e possibilidade de ter múltiplas conversas com o mesmo NPC).
- **Gestão de itens:** O jogador pode recolher e usar itens encontrados (diferentes tipos de itens, itens utilizáveis com efeitos específicos, validação de contexto de uso, e mensagens descritivas ao recolher/usar itens).

- **Consulta de Inventário e Pistas:** O jogador pode consultar itens recolhidos e pistas descobertas (Listagem organizada com descrições, sistema separado para itens e para informação, consulta de descrição detalhada de cada pista).
- **Sistema de Ajuda:** O jogador tem acesso a informações sobre comandos disponíveis (lista de todos os comandos disponíveis).
- **Encerramento do Jogo:** O jogador pode sair do jogo a qualquer momento. Usando o devido comando

2.3. Tecnologias Utilizadas

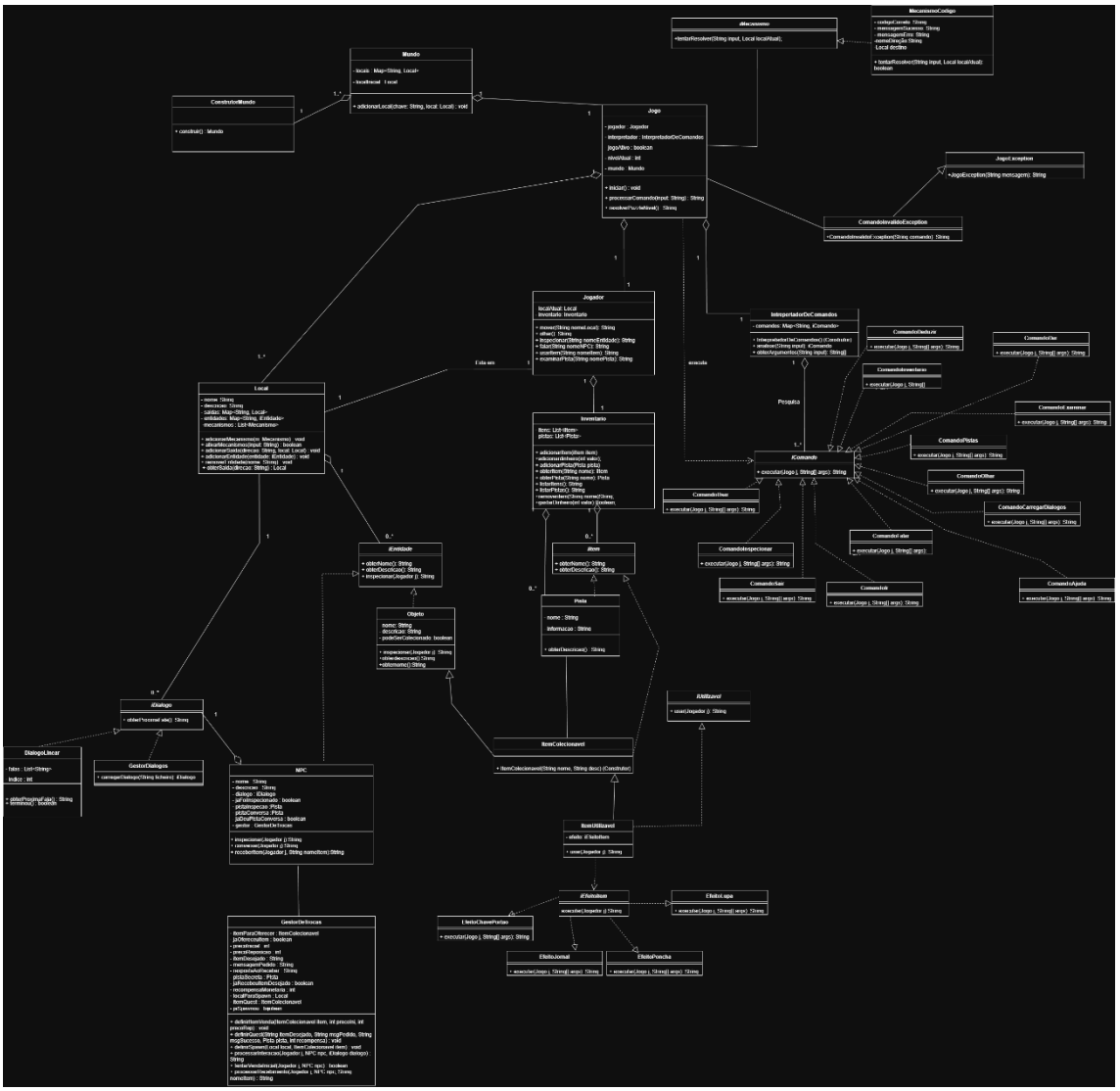
- **Linguagem:** Java
- **IDE:** NetBeans
- **Bibliotecas:** Biblioteca padrão do Java (java.util, java.io).
- **Ferramentas de Design:** Draw.io

2.4. Locais do Jogo:

- O jogo apresenta 9 locais distintos ambientados na Madeira, cada um com características únicas com diferentes elementos.
    - Zona Velha;
    - Mercado dos Lavradores;
    - Entrada do Porto;
    - Porto Industrial;
    - Painel de controle;
    - Armazém 4;
    - Portões da Quinta;
    - Corredor;
    - Gabinete;

3. Arquitetura do Sistema

3.1. Diagrama de Classes UML



(ABRIR LINK ANEXADO A IMAGEM DO UML)

O UML acima representa a estrutura completa do sistema, mostrando todas as classes, interfaces, atributos, métodos e seus relacionamentos. A arquitetura foi projetada seguindo os princípios SOLID para garantir modularidade, extensibilidade e maior facilidade de manutenção.

3.2. Descrição Detalhada das Classes

PACOTE: núcleo

- **Classe**: Jogo
- **Responsabilidade:** Controlador principal da aplicação. Mantem o loop de execução do jogo e coordena todos os componentes.
- **Atributos principais:**
  - jogador: Jogador - Referência ao jogador atual;
  - mundo: Mundo - Referência ao mundo do jogo;
  - interpretador: InterpretadorDeComandos - Processador de comandos;
  - emExecucao: boolean - Flag de controle do loop principal
  - nivelAtual: Referência a que nível o jogador se encontra;

  Métodos principais:
  - iniciar(): void - Inicializa o jogo e entra no loop principal;
  - processarComando(String): void - Processa comando digitado;
  - resolverPuzzelNivel(): void – processa o dilema daquele nível;

  Relações:
  - Associação com Jogador (1 para 1);
  -Associação com Mundo (1 para 1);
  - Associação com InterpretadorDeComandos (1 para 1);

- **Classe**:Mundo
- **Responsabilidade**: Construção e inicialização de todo o mundo do jogo. Cria todos os locais, itens, NPCs e suas interconexões.
- **Atributos principais:**
  - locais:Map<String,Local> - Mapa de todos os locais do jogo;
  - localInicial: Local - Referência ao local onde o jogo começa.

  Métodos principais:
  - construirMundo(): void - Método principal de construção.

  Classe: Local

  Responsabilidade: Representa um local que contém itens, NPCs e conexões com outros locais.

  Atributos principais:
  - nome: String - Nome identificador do local;
  - descricao: String - Descrição apresentada ao jogador;

  -saidas: Map<String, iEntidade> - Locais adjacentes;

  -mecanismos:List<Mecanismos>- Lista de mecanismos disponiveis

Métodos principais:
- adicionarMecanismos(m: Mecanismo) : void; , adicionar algum mecanismo.

- ativarMecanismo(input: String): boolean; ativar o mecanismo se verdadeiro

-adicionarSaida(entidade: iEntidade) : void; adicionar uma saída ao local

- removerEntidade(nome: String) : void; remover uma entidade

- obterSaida(direcao: String): Local. Obter a saída do locar

Relações:
-Associação com jogo (1 para 1).

Classe: Inventario

Responsabilidade: Gere os bens físicos (itens) e conhecimentos (pistas) do utilizador, bem como o saldo monetário.

Atributos principais:

- itens: List<iItem> - Lista de itens físicos no inventário;

- pistas: List<Pista> - Caderno de notas com informações descobertas;

- dinheiro: int - Saldo em euros do jogador.

Métodos principais:

- adicionarItem(iItem item): void - Guarda um item recolhido;

- removerItem(String nome): String- Remove um item (consumido/entregue);

- adicionarPista(Pista pista): void - Regista uma nova pista no caderno de notas;

- gastarDinheiro(int valor): boolean – remove a quantia de dinheiro .

- adicionarDinheiro(int valor): void - Adiciona dinheiro à carteira; - listarItens(): String - Gera lista textual do inventário; - listarPistas(): String - Gera lista textual das pistas descobertas.

Relações:

-Associação com Jogador (1 para 1).

3.3. Aplicação dos Princípios de POO

3.3.1. Encapsulamento

O encapsulamento foi aplicado rigorosamente em todo o projeto através do uso adequado de modificadores de acesso:

```
public class Jogador {
    private Local localAtual;
    private Inventario inventario;

    public Inventario obterInventario() { return inventario; }
    public Local obterLocalAtual() { return localAtual; }
```

Benefícios no uso de encapsulamento:

- Proteção da integridade dos dados;
- Alterações internas sem impacto em outras classes;
- Validação centralizada de regras de negócio;

3.3.2. interfaces

Interfaces foram utilizadas para ocultar detalhes complexos e expor apenas o essencial:

```
public interface iComando {
    String executar(Jogo jogo, String[] args);
}
```

```
public interface iEntidade {

    /**
     * Devolve o nome da entidade.
     *
     * @return O nome identificador.
     */
    String obterNome();

    /**
     * Devolve a descrição base da entidade.
     *
     * @return Texto descritivo.
     */
    String obterDescricao();

    /**
     * Define como a entidade reage ao ser examinada detalhadamente pelo jogador.
     *
     * @param j O jogador que realiza a inspeção.
     * @return O resultado da inspeção.
     */
    String inspecionar(Jogador j);
}
```

Benefícios na utilização de classes abstratas:

- Código mais limpo;
- Facilita manutenção e compreensão;
- Permite trabalhar com conceitos gerais;

3.3.3. Herança

A herança estabeleceu relações para ser mais fácil reutilização de código:

```
public class ItemColecionavel extends Objeto implements iItem {

    /**
     * Constrói um novo item colecionável.
     * Define automaticamente a propriedade 'podeSerColecionado' como verdadeira.
     *
     * @param nome      O nome do item.
     * @param descricao A descrição visual do item.
     */
    public ItemColecionavel(String nome, String descricao) {
        super(nome, descricao, true);
    }
}
```

```
public class ItemUtilizavel extends ItemColecionavel implements iUtilizavel {

    /**
     * A estratégia que define o que acontece quando o item é usado.
     */
    private iEfeitoItem efeito;

    /**
     * Constrói um item utilizável com um efeito específico.
     *
     * @param nome      O nome do item.
     * @param descricao A descrição do item.
     * @param efeito    A implementação da lógica de uso (Strategy).
     */
    public ItemUtilizavel(String nome, String descricao, iEfeitoItem efeito) {
        super(nome, descricao);
        this.efeito = efeito;
    }
```

Benefícios do uso de herança:
- Eliminação de código duplicado;
- Organização hierárquica clara;
- Facilita extensão do sistema **;**

3.3.4. Polimorfismo

O polimorfismo permitiu tratar objetos de diferentes classes de forma igual:

```
// ITENS COM EFEITOS (Usando as classes concretas do pacote itens)
ItemColecionavel lupa = new ItemUtilizavel("Lupa", "Vidro riscado, mas funcional.", new EfeitoLupa());
ItemColecionavel jornal = new ItemUtilizavel("Jornal", "Jornal encharcado com noticias de crimes.", new EfeitoJornal());
ItemColecionavel poncha = new ItemUtilizavel("Poncha", "Ouro liquido da Madeira.", new EfeitoPoncha());
```

Benefícios do uso de herança:
- Código mais flexível e extensível;
- Redução de estruturas complexas;
- Facilita adição de novos tipos**;**

3.4. Aplicação dos Princípios SOLID

S - **Single Responsibility Principle (Princípio da Responsabilidade Única)**, todas as nossas classes respeitam o **S,** como por exemplo a classe inventário trata só do inventario, a classe jogador só tem responsabilidade pelas funções do jogador,etc.

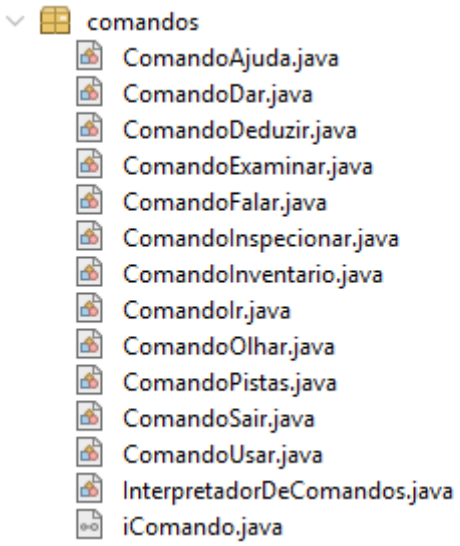
,

Benefícios no projeto:
-Fácil localização de bugs;
-Testes mais simples pois cada classe testa uma coisa;
-Mudanças isoladas ,alterar diálogos não afeta comandos por exemplo.

O - Open/Closed Principle (Princípio Aberto/Fechado)
Classes abertas para extensão, fechadas para modificação:

package comandos;

// Sistema ABERTO para novos comandos



// Adicionar ao interpretador SEM modificar código existente

Benefícios no projeto:
-Adição de características sem risco de quebrar o código existente;
-Sistema cresce de forma controlada.

L - Liskov Substitution Principle (Princípio da Substituição de Liskov)
Subclasses podem substituir superclasses sem quebrar funcionalidade:

```
public class ItemColecionavel extends Objeto implements iItem {

    /**
     * Constrói um novo item colecionável.
     * Define automaticamente a propriedade 'podeSerColecionado' como verd
     *
     * @param nome      O nome do item.
     * @param descricao A descrição visual do item.
     */
    public ItemColecionavel(String nome, String descricao) {
        super(nome, descricao, true);
    }
}
```

Benefícios no projeto:
-Código previsível e confiável;
-Polimorfismo funciona corretamente;
-Hierarquias bem projetadas;

I - Interface Segregation Principle (Princípio da Segregação de Interfaces)
Interfaces específicas ao invés de genéricas:

```
public interface iComando {
    String executar(Jogo jogo, String[] args);
}
```

Benefícios no projeto:
-Classes não são forçadas a implementar métodos que não usam;
-Interfaces mais coesas e focadas.

D - Dependency Inversion Principle (Princípio da Inversão de Dependência)
Depender de abstrações, não de implementações concretas:

```
public interface iDialogo {

    /**
     * Obtém a próxima linha de texto disponível na sequência do diálogo.
     *
     * @return A String contendo a fala do personagem.
     */
    String obterProximaFala();

    /**
     * Verifica se o diálogo chegou ao fim da sua sequência de falas.
     *
     * @return true se não houver mais falas novas, false caso contrário.
     */
    boolean terminou();
}
```

Benefícios no projeto:
-Fácil substituição de implementações;
-Flexibilidade para mudanças.

4. Decisões de Projeto e Justificações

4.1. Padrão Command

Decisão: Utilização do Design Pattern Command para gerir as ações do jogador. **Justificação:** Em vez de utilizar uma estrutura condicional gigante (if/else ou switch) na classe principal para verificar o que o jogador digitou, cada comando foi encapsulado numa classe própria. Isso facilita a manutenção e permite que múltiplos desenvolvedores trabalhem em comandos diferentes simultaneamente.

Justificativa da escolha:
O Pattern Command foi escolhido pelos seguintes motivos:
1. **Manutenibilidade:** Cada comando em sua própria classe facilita localização de bugs;
2. **Testabilidade:** Cada comando pode ser testado isoladamente
3. **Extensibilidade:** Adicionar novo comando não requer modificar código existente (OCP)
4. **Responsabilidade Única:** Cada classe de comando tem apenas uma responsabilidade (SRP)

4.2. Hierarquia de Exceções Personalizadas (extends Exception)

Decisão: Criação da classe JogoException herdando diretamente de java.lang.Exception.

Justificação: Optou-se por as nossas proprias exceptions para obrigar o tratamento de erros previsíveis dentro da lógica do jogo. Ao contrário de lançar uma exceção genérica (RuntimeException), a JogoException permite capturar erros de regra de jogo (ex: inserir um comando que não exista) e exibir uma mensagem informativa do erro ao utilizador, sem encerrar o programa.

4.3. Separação em Pacotes

Decisão: Organização do código em pacotes .

Justificação:
1. Coesão: Classes relacionadas agrupadas;
2. Navegação: Fácil encontrar classes específicas;
3. Escalabilidade: Novos elementos adicionados ao pacote correto;
4. Princípio da Responsabilidade Única: Cada pacote tem propósito claro e único.

4.4. Gestão de Pistas /Inventário
Decisão: Separar completamente o sistema de pistas do inventário de itens.

Porque tomamos esta decisão? Inicialmente, consideramos tratar pistas como itens normais no inventário. Porém, isso criava confusão pois pistas são informações e itens podem ser utilizáveis ou colecionáveis .

Justificação:
1. **Clareza:** Separação lógica entre físico e informação;
2. **Sem limite de pistas:** Informações não ocupam espaço no inventário.

4.5. Desafios Enfrentados

Desafio 1: Gestão de Referências Circulares entre Locais

Problema: Locais precisam referenciar uns aos outros (ex: Zona Velha ↔ Mercado), criando potencial de referências redundantes.

Desafio 2: Parsing de Comandos com Nomes Compostos

Problema: Comandos como ir zona velha têm espaços no parâmetro.

```java
/**
 * Utilitário para separar os argumentos da palavra-chave do comando.
 * Remove a primeira palavra (o comando) e devolve o resto como um array.
 *
 * @param input A linha de texto completa.
 * @return Array contendo apenas os argumentos (ex: se input for "ir zona velha", devolve ["zona velha"]).
 */
public String[] obterArgumentos(String input) {
    String[] partes = input.trim().split(" ");
    if (partes.length > 1) return Arrays.copyOfRange(partes, 1, partes.length);
    return new String[0];
}
```

Desafio 3: Validação de Input do Utilizador

Problema: Utilizadores digitam qualquer coisa (espaços extras, Enter vazio, CAPS LOCK).

```java
/**
 * Analisa a string de entrada, extrai a palavra-chave e devolve o comando correspondente.
 *
 * @param input A linha de texto digitada pelo jogador.
 * @return A instância do comando correspondente à palavra-chave.
 * @throws ComandoInvalidoException Se a primeira palavra não corresponder a nenhum comando registado.
 */
public iComando analisar(String input) throws ComandoInvalidoException {
    String[] partes = input.trim().split(" ");
    String chave = partes[0].toLowerCase();

    if (comandos.containsKey(chave)) {
        return comandos.get(chave);
    }

    // LANÇAR A EXCEÇÃO PERSONALIZADA
    throw new ComandoInvalidoException(chave);
}
```

5. Guia para Expansão e Modificação

5.1. Pontos de Extensibilidade

O sistema foi projetado com múltiplos pontos de extensão que permitem adicionar funcionalidades sem modificar código existente::

1. Novos comandos (através de iComando)
2. Novos tipos de itens (herança de Item)
3. Novos locais (instanciação no Mundo)
4. Novos NPCs (instanciação no Mundo)
5. Novos comportamentos de itens (interfaces como iUtilizavel)

5.2. Adicionando um Novo Comando

Para criar, por exemplo, um comando "saltar":

1. Crie a classe ComandoSaltar no pacote comandos.
2. Implemente a interface iComando.
3. No método executar, defina a lógica (ex: imprimir uma mensagem).
4. Registe a palavra-chave "saltar" no mapa do InterpretadorDeComandos.
5. Adicionar à lista de ajuda.

5.3. Adicionando Novos Locais

1. Crie o local em Mundo.java.
2. Conectar com locais existentes.
3. Adicionar itens ao local.
4. Adicionar NPCs se assim o quiser.

5.4. Boas Práticas para Continuação do Desenvolvimento

1. Manter a separação em pacotes;
2. Implementar interfaces ao invés de herdar de classes concretas;
3. Usar exceções personalizadas para erros lógicos;
4. Fazer sempre documentação do código;

5. Quando adicionado alguma funcionalidade nova ,fazer testes antes de adicionar a próxima;

5.5. Potenciais Melhorias Futuras

- Sistema de Save/Load;
- Sistema de Conquistas;
- Múltiplas Histórias;
- Novos comandos;
- Novos NPCs;
- Diálogo do utilizador diretamente não por comandos;

6. Testes e Validação

6.1. Estratégia de Testes

O projeto foi testado através de **testes manuais funcionais**, focando na experiência do utilizador e na validação de requisitos funcionais. A estratégia de testes seguiu três níveis:

-Nível 1: Testes Unitários Manuais

Teste de cada comando individualmente;
Validação de comportamento esperado;
Verificação de mensagens de erro.

-Nível 2: Testes de Integração

Teste de fluxos completos (sequência de comandos);
Validação de interações entre componentes.

-**Nível 3: Testes de Aceitação**

Jogabilidade completa do início ao fim;
Verificação da experiência da perspetiva do utilizador;
Validação do objetivo final que é resolver o mistério.

6.2. Cenários de Teste Implementados

-1: Navegação entre Locais

Objetivo: Verificar movimento correto entre todos os locais
Casos testados:
Movimento para local válido;
Tentativa de movimento para local inexistente;
Tentativa de acesso a local bloqueado;
Retorno ao local anterior;

-2: Sistema de Inventário
Objetivo: Validar gestão de itens
Casos testados:
Apanhar item disponível;
Tentar apanhar item inexistente;

-3: Interação com NPCs
Objetivo: Testar sistema de diálogos
Casos testados:
Falar com NPC disponível;
Tentar falar com NPC inexistente;

-**4: Sistema de Pistas**
Objetivo: Verificar coleta e consulta de pistas
Casos testados:
Adicionar pista através de diálogo;
Adicionar pista através de inspeção;
Consultar caderno de pistas;
Verificar não duplicação de pista.

-**5: Interpretador de Comandos**
Objetivo: Testar o programa
Casos testados:
Comando válido minúsculas;
Comando válido MAIÚSCULAS;
Comando com espaços extras;
Comando vazio (apenas Enter);
Comando inexistente;
Comando com parâmetros compostos ("ir zona velha").

6.3. Correção de Erros

Durante o desenvolvimento, foi identificado um erro onde o InterpretadorDeComandos falhava se o utilizador introduzi se apenas espaços ou Enter. O erro foi corrigido adicionando uma verificação trim().isEmpty() antes da execução do comando.

7. Conclusão

7. CONCLUSÃO

7.1. Resultados Alcançados

O projeto "Sherlock Holmes - Jogo de Investigação em Texto" foi concluído com sucesso, atingindo todos os objetivos propostos no enunciado:

Requisitos Funcionais Implementados: Sistema de navegação entre 9 locais interligados, observação detalhada de ambientes, inspeção de objetos com descoberta de pistas, diálogos interativos com NPCs, sistema de, gestão de pistas separadas do inventário, interpretador de comandos, sistema de ajuda caso o utilizador precise e objetivo final (resolver mistério) implementado..

A Qualidade do Código está composta nestes seguintes fatores**:** Organização em pacotes funcionais, tratamento robusto de exceções, código comentado e documentado.

7.2. Aprendizagens Obtidas

O desenvolvimento deste projeto proporcionou aprendizagens significativas em múltiplas áreas:

1. Design Patterns: Compreensão prática do Command Pattern e sua aplicação real. Percebemos como patterns resolvem problemas recorrentes de forma elegante e estruturada.

2. Princípios SOLID: A aplicação consciente dos princípios SOLID desde o início do projeto facilitou imensamente a manutenção e extensão do código. Particularmente o Open/Closed Principle.

3. Tratamento de Exceções: Aprendemos a diferença prática entre checked e unchecked exceptions, e quando usar cada uma.

4.Estruturas de Dados: Escolha apropriada entre List, Set, Map baseado nas necessidades de cada contexto.

Por exemplo, Map para locais permitiu busca dos locais enquanto List para inventário manteve ordem de coleta dos itens.

Este projeto representou uma aplicação prática e abrangente dos conceitos de Programação Orientada a Objetos, indo além da teoria estudada nas aulas. A criação de um jogo completo e funcional demonstrou como os princípios SOLID e boas práticas de POO não são apenas conceitos académicos, mas ferramentas

essenciais para desenvolvimento de software de qualidade. O principal desafio foi equilibrar a complexidade técnica, aplicação correta de SOLID, com a simplicidade do uso. Conseguimos criar uma arquitetura sofisticada mantendo uma interface simples para o jogador.

Agradecemos aos docentes da disciplina pelas orientações fornecidas durante o desenvolvimento e a ajuda que forneceram.